

OpenClaw / Multica Multi-Agent Guide

System-Native Implementation & Troubleshooting across Linux, macOS, and Windows

NATIVE DEPLOYMENT

PM2 PROCESS MANAGEMENT

CROSS-PLATFORM

MASKED & SANITIZED

1. Executive Overview & Native Architecture

This guide details the complete procedure for deploying and running multiple simultaneous **OpenClaw / Multica** autonomous agents directly on a host operating system without containerization (Docker). Running agents natively provides direct OS access, lower memory overhead, and simplified hardware integration, but requires explicit environment isolation and strict process management.

Core Native Deployment Architecture

Each native agent operates as an isolated background process with its own dedicated configuration file (`openclaw.json`), unique listening port, isolated workspace directory, and separate environment variables.

Agent Identifier	Assigned Port	Workspace Directory Path	Process Manager Instance
Agent-Alpha (e.g., Nova)	8081	<code>~/.openclaw/agents/nova/workspace</code>	<code>openclaw-nova</code>
Agent-Beta (e.g., Sage)	8082	<code>~/.openclaw/agents/sage/workspace</code>	<code>openclaw-sage</code>
Agent-Gamma (e.g., Iris)	8083	<code>~/.openclaw/agents/iris/workspace</code>	<code>openclaw-iris</code>
Agent-Delta (e.g., Rex)	8084	<code>~/.openclaw/agents/rex/workspace</code>	<code>openclaw-rex</code>

2. System Prerequisites & Installation

Ensure system binaries and dependencies are installed natively on the host platform before configuring individual agents.

Linux (Ubuntu / Debian)

```
# Update system packages and install Node.js (v18+), Git, and build tools
sudo apt-get update && sudo apt-get install -y curl git build-essential
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt-get install -y nodejs

# Install PM2 globally for background process supervision
sudo npm install -g pm2 openclaw
```

macOS (Homebrew)

```
# Install Node.js, Git, and PM2 via Homebrew / NPM
brew install node git
npm install -g pm2 openclaw
```

Windows (PowerShell Admin / WSL2)

```
# Install via Winget or standard Node.js installer
winget install OpenJS.NodeJS.LTS
winget install Git.Git
```

```
# Install PM2 process manager globally
npm install -g pm2 openclaw
```

3. Step-by-Step Multi-Agent Native Setup

Step 1: Directory Structure Setup

Create dedicated isolated directories for each agent's configuration and working filesystem.

```
# Linux / macOS Directory Creation
mkdir -p ~/.openclaw/agents/{nova,sage,iris,rex}/workspace

# Windows PowerShell Directory Creation
New-Item -ItemType Directory -Path "$env:USERPROFILE\.openclaw gents
ova\workspace" -Force
New-Item -ItemType Directory -Path "$env:USERPROFILE\.openclaw gents\sage\workspace" -Force
New-Item -ItemType Directory -Path "$env:USERPROFILE\.openclaw gents\iris\workspace" -Force
New-Item -ItemType Directory -Path "$env:USERPROFILE\.openclaw gents
ex\workspace" -Force
```

Step 2: Creating Agent Configuration Files (openclaw.json)

Every agent must have a distinct `openclaw.json` file located in its respective agent root directory. Sensitive credentials (API keys, Telegram bot tokens) are injected via environment variables or masked placeholders.

Important JSON Syntax & Security Rule

Ensure valid JSON formatting without trailing commas or unescaped path backslashes. All access tokens must be managed securely and kept out of public repositories.

Example Configuration (Agent Nova - `~/.openclaw/agents/nova/openclaw.json`):

```
{
  "agent": {
    "name": "Nova",
    "id": "nova-01",
    "workspace": "./workspace"
  },
  "server": {
    "port": 8081,
    "host": "127.0.0.1"
  },
  "integrations": {
    "telegram": {
      "enabled": true,
      "bot_token": "ENV_AGENT_NOVA_TELEGRAM_TOKEN"
    }
  },
  "tools": {
    "git": true,
    "bash": true,
    "system_cli": true
  }
}
```

Repeat this structure for **Sage** (Port `8082`), **Iris** (Port `8083`), and **Rex** (Port `8084`), altering the agent name, ID, and listening port accordingly.

Step 3: Environment Variable Configuration

Create an environment initialization script or `.env` file to hold masked tokens safely.

```
# File: ~/.openclaw/agents/env_config.sh
export ENV_AGENT_NOVA_TELEGRAM_TOKEN=""
export ENV_AGENT_SAGE_TELEGRAM_TOKEN=""
export ENV_AGENT_IRIS_TELEGRAM_TOKEN=""
export ENV_AGENT_REX_TELEGRAM_TOKEN=""
export OPENCLAW_GLOBAL_SECRET=""
```

4. Process Management with PM2

To run agents as resilient host services that restart automatically on system reboot or crashes, construct a PM2 ecosystem file (`ecosystem.config.js`).

```
// File: ~/.openclaw/ecosystem.config.js
module.exports = {
  apps: [
    {
      name: 'openclaw-nova',
      script: 'openclaw',
      args: 'start --config ~/.openclaw/agents/nova/openclaw.json',
      env: {
        NODE_ENV: 'production',
        TELEGRAM_TOKEN: process.env.ENV_AGENT_NOVA_TELEGRAM_TOKEN
      }
    },
    {
      name: 'openclaw-sage',
      script: 'openclaw',
      args: 'start --config ~/.openclaw/agents/sage/openclaw.json',
      env: {
        NODE_ENV: 'production',
        TELEGRAM_TOKEN: process.env.ENV_AGENT_SAGE_TELEGRAM_TOKEN
      }
    },
    {
      name: 'openclaw-iris',
      script: 'openclaw',
      args: 'start --config ~/.openclaw/agents/iris/workspace/openclaw.json',
      env: {
        NODE_ENV: 'production',
        TELEGRAM_TOKEN: process.env.ENV_AGENT_IRIS_TELEGRAM_TOKEN
      }
    },
    {
      name: 'openclaw-rex',
      script: 'openclaw',
      args: 'start --config ~/.openclaw/agents/rex/openclaw.json',
      env: {
        NODE_ENV: 'production',
        TELEGRAM_TOKEN: process.env.ENV_AGENT_REX_TELEGRAM_TOKEN
      }
    }
  ]
};
```

Launching and persisting processes:

```
# Load environment variables and start all agents
source ~/.openclaw/agents/env_config.sh
pm2 start ~/.openclaw/ecosystem.config.js

# Save PM2 process list and configure auto-start on boot
pm2 save
pm2 startup
```

5. Troubleshooting, Obstacles & Root Cause Analysis

During prior setup iterations, several operational obstacles were identified. The table below documents these failure points and their exact resolution in a native environment.

Obstacle / Error Type	Observed Behavior & Cause	Native Fix & Mitigation
JSON Configuration Syntax Error	Agent process immediately crashes upon execution with Unexpected token or SyntaxError . Caused by trailing commas, missing quotes, or unescaped backslashes in paths on Windows.	Validate all openclaw.json files using a JSON validator prior to starting (python3 -m json.tool openclaw.json). Use forward slashes (/) in paths across all platforms including Windows.
Port Collision (EADDRINUSE)	Secondary agents fail to initialize server with Error: listen EADDRINUSE: address already in use :::8081 .	Ensure each agent configuration specifies a distinct port (e.g., 8081, 8082, 8083, 8084). Verify active port bindings using lsof -i :8081 (Linux/macOS) or Get-NetTCPConnection - LocalPort 8081 (Windows).
Missing Native Binary Dependencies	Agent skills requiring Git or shell integration fail with 'git' is not recognized as an internal or external command or binary missing errors.	In native execution, agents rely on the host system PATH . Verify binary availability on the system host by running git --version , curl --version , and python3 --version in the exact shell running PM2.
Telegram Bot Pairing & Approval Timeouts	Bot receives messages but agent does not process actions. Log displays pairing authorization required or token auth failures.	1. Verify each agent is bound to a unique Telegram bot token (do not share tokens across agents). 2. Run initial pairing interactively in foreground mode (openclaw pair --config <path>) to approve incoming chat IDs before delegating to PM2.
Dashboard Token Authentication Expiry	Web UI dashboard returns 401 Unauthorized or token mismatch when navigating between agent interfaces.	Generate permanent API session tokens in openclaw.json using "auth_token": "MASKED_STATIC_TOKEN" rather than relying on ephemeral dynamic session keys. Ensure individual dashboard ports do not conflict.
File & Folder Permission Denied	Agent fails to write persistent state or logs to ./workspace directory.	Grant read/write ownership to the active runtime user: chmod -R 755 ~/.openclaw (Linux/macOS). On Windows, ensure PowerShell is executed without restrictive execution policy blocking local scripts.

6. Cross-Platform Command Matrix

Action	Linux (Ubuntu/Debian)	macOS	Windows (PowerShell)
Start All Agents	<code>pm2 start ecosystem.config.js</code>	<code>pm2 start ecosystem.config.js</code>	<code>pm2 start ecosystem.config.js</code>
Check Status	<code>pm2 status</code>	<code>pm2 status</code>	<code>pm2 status</code>
View Live Logs	<code>pm2 logs openclaw-nova</code>	<code>pm2 logs openclaw-nova</code>	<code>pm2 logs openclaw-nova</code>
Inspect Port Usage	<code>sudo netstat -tulpn grep 808</code>	<code>lsof -i :8081-8084</code>	<code>Get-NetTCPConnection - LocalPort 8081</code>
Restart Single Agent	<code>pm2 restart openclaw-sage</code>	<code>pm2 restart openclaw-sage</code>	<code>pm2 restart openclaw-sage</code>
Stop All Services	<code>pm2 stop all</code>	<code>pm2 stop all</code>	<code>pm2 stop all</code>

7. Security & Sanitization Summary

When sharing or deploying multi-agent guides and configurations across teams:

- **Redact All Private Tokens:** Always replace Telegram Bot API tokens, OpenClaw secret keys, and personal credentials with standardized placeholders (e.g., `<MASKED_TELEGRAM_TOKEN>`).
- **Restrict File Permissions:** Apply `chmod 600` to environment config files containing secrets to prevent unauthorized access by local non-root users.
- **Bind to Localhost:** For internal multi-agent coordination not exposed to the public internet, bind HTTP listeners to `127.0.0.1` rather than `0.0.0.0` .